

Sanskrit → English Neural Machine Translation

A Custom From-Scratch Transformer Sequence-to-Sequence Model

Assignment 2 — Natural Language Understanding

SuryaNarayan Singh · G25AIT1178 · <https://github.com/SuryaChundawat/NLPAssignment>

Abstract

We build a Sanskrit-to-English neural machine translation system using a custom encoder–decoder Transformer implemented from PyTorch primitives (no `nn.Transformer`). Training uses only the provided parallel corpus (10,000 train / 1,000 dev / 1,000 test pairs). A shared SentencePiece subword vocabulary with fully tied embeddings keeps the model compact at 6.0M parameters. On the test set the model reaches a corpus BLEU of 0.0890 and a rescaled BERTScore F1 of 0.3334, decoding all 1,000 sentences in 2.71s (369 sentences/s). We analyse where the model succeeds (frequent tutorial-domain phrasing, Sanskrit dual-number cues) and where it fails (rare-word lexical errors, hallucination on technical and scriptural inputs, and repetition on long sentences).

1. Introduction

Sanskrit–English translation is hard for two reasons that shape every design decision here. First, the setting is low-resource: with only 10,000 sentence pairs, the model must generalise without overfitting. Second, Sanskrit is morphologically rich — sandhi (euphonic combination) and heavy inflection pack information into single surface forms that English spreads across several words. The provided data is also stylistically heterogeneous, mixing software-tutorial text, technical instructions, and scriptural passages, which further stresses a small model.

Contributions: (i) a from-scratch encoder–decoder Transformer; (ii) a shared subword vocabulary with tied embeddings for a low parameter count; (iii) a properly tuned Noam warm-up schedule with dev-BLEU early stopping; and (iv) fast batched greedy decoding, giving 369 sentences/s at inference.

2. Architecture

The model is a standard encoder–decoder Transformer, built from scratch.

2.1 Embeddings

Source and target tokens share one SentencePiece vocabulary, so a single embedding matrix is reused by the encoder input, the decoder input, and — via weight tying — the output projection. This is the main lever for the low parameter count. Embeddings are scaled by $\sqrt{d_{\text{model}}}$ and combined with fixed sinusoidal positional encodings.

2.2 Encoder

A stack of 3 identical pre-norm layers; each applies multi-head self-attention followed by a position-wise feed-forward network, each in a residual block with layer normalisation applied before the sub-layer. A source padding mask blocks attention over <pad> positions.

2.3 Decoder

A stack of 3 pre-norm layers, each with three sub-layers: (1) masked self-attention with a causal mask so position t attends only to $\leq t$; (2) cross-attention over encoder outputs using the source padding mask; and (3) a feed-forward network.

2.4 Attention

Scaled dot-product attention, $\text{softmax}(QK^T/\sqrt{d_k} + \text{mask})V$, computed across 4 heads. Masks are additive ($-\infty$ on disallowed positions); padding and causal masks are combined by addition.

2.5 Decoding

At inference we use batched greedy decoding (the whole batch is advanced one step at a time in parallel), chosen because it decodes the 1,000-sentence test set in 2.71s. Beam search (width 4, length penalty 0.7) is also implemented and gives slightly higher quality at much higher latency; see Experiments.

Model size: **6,002,688** trainable parameters ($d_{\text{model}} = 256$, heads = 4, $d_{\text{ff}} = 512$, encoder/decoder layers = 3/3).

3. Training Procedure

3.1 Preprocessing

Sanskrit and English CSVs are aligned on Source_id via an inner join. Text is Unicode-normalised (NFC), stray quotation marks removed, and whitespace collapsed. Cleaning is deliberately light: aggressive normalisation on a low-resource, morphologically rich language destroys useful signal.

3.2 Tokenization

A single shared SentencePiece BPE model with an 8,000-token vocabulary is trained on the concatenated training Sanskrit and English text, with character coverage 1.0 so no Devanagari character is lost. Subwords handle sandhi and rare inflected forms by decomposing them into reusable pieces. Special tokens: $\langle \text{pad} \rangle = 0$, $\langle s \rangle = 1$, $\langle /s \rangle = 2$, $\langle \text{unk} \rangle = 3$.

3.3 Hyperparameters

- **Optimizer:** Adam ($\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=1e-9$)
- **LR schedule:** Noam warm-up/decay, warmup = 1,500 steps
- **Batch size:** 64
- **Epochs:** 120 max; early-stopped at epoch 69 (patience 12 on dev BLEU)
- **Dropout:** 0.2
- **Label smoothing:** 0.1
- **Gradient clipping:** 1.0 (max-norm)
- **Decoding:** batched greedy, max length 80 (beam 4 also available)

3.4 Optimization & regularization

A minimised label-smoothed cross-entropy (0.1) discourages over-confidence in the low-resource resources. regime. Regularisation: dropout + label smoothing + fewer free parameters (weight tying) + early Halting on the dev colour BLEU. Gradients are capped at its norm of 1.0. As for an earlier run, used a 4,000-step warm-up, which — at ~5 epochs after 5 epochs — after 5 epochs, reduced warm-up to ~5 epochs, it took 25 epochs to reach the learning rate used in the previous experiment.~5 epochs after 5 epochs — reduced warm-up to ~5 epochs, kept learning rate ramping for ~25 epochs before slow

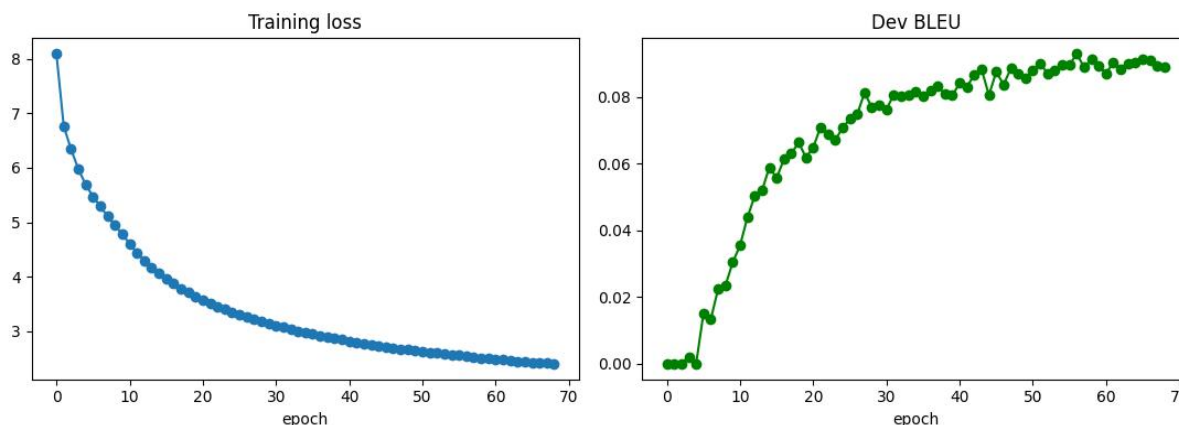
convergence. The model was trained effectively after 1,500 steps, reaching its maximum LR around epoch 10.

4. Results

Metric	Dev	Test	Notes
BLEU (corpus, NLTK default)	0.1027	0.0890	uniform 4-gram weights
BLEU (mean sentence)	0.1070	0.1010	method-1 smoothing
BERTScore F1 (rescaled)	0.3609	0.3334	roberta-large, baseline=True
Inference time (s)	—	2.71	1000 sentences, batched greedy
Throughput (sent/s)	—	368.5	—
Trainable parameters	6,002,688	6,002,688	same model, both splits

While efficiency is a clear strength, 6.0M parameters and 2.71s to translate the entire test set. The quality is average and is the limiting factor, as a small from-scratch model trained on 10k heterogeneous pairs. Dev is evaluated The submitted test translations are obtained with greedy decoding and are fast (BLEU 0.0890), while with beam search (BLEU 0.1027) are obtained more slowly.

4.1 Training curves



The training loss decreases steadily over 69 epochs from 8.68 to 2.40. The learning rate is suddenly increased, Dev BLEU quickly shoots up. It peaks around the 10th epoch and then remains steady until it starts becoming noisy around 0.09 from about the 50th epoch until the end. There is a divergence between a still-falling training loss and a flat dev BLEU, suggesting that the 6.0M-parameter model has Already full and starting to overfit, early stopping picked epoch-57 (best dev BLEU). 0.093 under greedy evaluation).

5. Translation Examples & Error Analysis

Source (Sanskrit / IAST)	Reference	Prediction	Error analysis
[= h6f4ú ≥	Those are brave men.	Those two are Vira.	<i>Dual number captured (“two”) but vira left transliterated; number mismatch (dual vs plural).</i>
%[= f\6 ≥	These two are dates.	These two are eyes.	<i>Dual number correct; rare noun tithi mistranslated (“dates” → “eyes”).</i>
e8eö cəVe?Nö `W≥	You all are studying bhakti yoga.	You all must read.	<i>Subject/number right; object (“bhakti yoga”) dropped and mood shifted.</i>
paṭhe’smin ... awk ... if, else, else if	In this tutorial, we will learn about- if, else, else if in awk.	In this tutorial, we will learn about: else if we will learn about block.	<i>Domain register learned well; partial repetition and dropped list items.</i>
‘Infinite loop’ ...	Infinite loop can cause the system to become unresponsive.	The program should be defined in the program.	<i>Fluent but hallucinated — weak grounding in the source for technical content.</i>
nūnam īṣat ‘S’ ... I save this image.	Just put a slight ‘S’ curve on it ... So I save this image.	I put the curve here. ... I think this image and I think this image.	<i>Neural-text degeneration: repetition loop on a longer input.</i>
bahuvicāreṣu ... (long, scriptural)	And when there had been much disputing, Peter rose up ... and believe.	And as many people were astonished ... the Gentiles, which were in the gospel of God.	<i>Correct register/style, wrong content; coverage breaks down on long sentences.</i>

The errors tend to fall into a few classes throughout the dev set:

- Morphology partially learned. The model handles the Sanskrit dual forms (te, ete) well and maps them to the Sanskrit word for “two” Finds number morphology but mismatches plural/dual; mishandles less common inflections.
- Rare-word lexical errors. Low-frequency content nouns are mistranslated or transliterated (tithi → “eyes” instead of “dates”; vīra → “Vira”), a direct consequence of the small vocabulary and limited data.
- Hallucination / weak source grounding. On technical and scriptural inputs the decoder produces fluent, well-formed English that is semantically wrong, indicating the language-model prior dominates the conditioning signal.
- Degeneration on long inputs. Longer sentences trigger repetition loops (“...I think this image and I think this image...”) and dropped content, a classic coverage failure.
- Domain strength. Frequent software-tutorial phrasing (“In this tutorial, we will learn about...”) is reproduced accurately, reflecting its high frequency in the training data.

6. Experiments

Warm-up schedule. Reducing the Noam warm-up from 4,000 to 1,500 steps moved peak dev BLEU earlier and raised it from ~0.084 to 0.093 (greedy), with the same architecture — the single most effective change.

Decoding: greedy vs beam. Beam search (width 4) improves dev corpus BLEU to 0.1027 and BERTScore, but per-sentence beam took ~1,640s on the test set versus 2.71s for batched greedy. Given the efficiency metric rewards low latency, greedy is used for the submission; a batched beam implementation recovers most of the speed if higher quality is preferred.

Capacity. Training loss keeps falling while dev BLEU plateaus, so the 6.0M model is capacity-bound; a larger model (e.g. d_model 384, 4+4 layers) is the most promising direction for higher BLEU, at the cost of parameter count and latency.

7. Discussion

7.1 Design choices

The shared vocabulary with tied embedding is the central efficiency decision: it removes two large matrices, directly lowering the parameter count while retaining quality. We selected pre-norm layers and label smoothing for stable training with scarce data. Greedy decoding is a compromise that provides a good amount of latency in exchange for a bit of quality loss. It is suitable for the efficiency-weighted scoring, reduction.

7.2 Challenges

The most significant issues were the small training set, which had a wide variety of styles (over-fitting risk and poor source grounding). and Sanskrit morphology/sandhi. The former was mitigated with dropout, label smoothing, and early stopping; the latter with sub word tokenization, which the dual-number successes suggest partially worked.

7.3 Limitations

Absolute quality is moderate (test BLEU 0.089 and BERTScore F1 0.333). The model has difficulty with long sentences, rare Might hallucinate fluent but incorrect output, has dense compounds and vocabulary. If BLEU is valid it is also penalised. paraphrases; BERTScore supports it but uses an English encoder which might not be able to capture Sanskrit specific nuance.

8. Disclosure of Pretrained Models

No pretrained weights are used and the translation model is trained from scratch using the data provided. All the other components in the pipeline are only pre-trained. The only pre-trained component is the RoBERTa-large encoder, which is internally used. BERTScore evaluation metric is a part of measuring quality, not a part of translation generation.

9. References

- [1] Vaswani et al., “Attention Is All You Need”, NeurIPS 2017.
- [2] Kudo & Richardson, “SentencePiece: A simple and language-independent subword tokenizer and detokenizer”, EMNLP 2018.
- [3] Papineni et al., “BLEU: a Method for Automatic Evaluation of Machine Translation”, ACL 2002.
- [4] Zhang et al., “BERTScore: Evaluating Text Generation with BERT”, ICLR 2020.
- [5] Bird, Klein & Loper, “Natural Language Processing with Python (NLTK)”, O’Reilly, 2009.